

Ch.2 Ollama LLM 활용의 기본 개념 (LangChain) — 학습 자료

수업 목표

- Ollama 로 로컬 LLM 을 돌리는 방법을 익힌다.
- Ollama, OpenAI, Anthropic 등 여러 LLM 벤더를 LangChain 으로 통일된 방식으로 호출하는 법을 익힌다.
- LangChain 의 프롬프트 템플릿(PromptTemplate, ChatPromptTemplate) 사용법을 익힌다.
- 출력 파서(StrOutputParser, JsonOutputParser, Pydantic 구조화 출력)로 원하는 형식의 답을 받는 법을 익힌다.
- LCEL(파이프 연산자 |)로 여러 단계를 체인으로 연결하는 법을 익힌다.

사전 준비사항

항목	내용
Ollama 설치	ollama.com 에서 다운로드 → 설치
모델 다운로드	ollama pull deepseek-r1:1.5b, ollama pull llama3.2:1b, ollama pull exaone3.5:2.4b 등 (Window 키+R → powershell)
패키지 설치	pip install langchain-ollama langchain-openai langchain-anthropic python-dotenv pydantic
API 키	OpenAI 는 OPENAI_API_KEY, Anthropic 은 ANTHROPIC_API_KEY 환경변수 필요 (.env 파일 + load_dotenv())

1. LLM 을 활용하여 답변 생성하기

1) Ollama 를 이용한 로컬 LLM 이용

요약: Ollama 는 로컬 PC 에서 오픈소스 LLM 을 실행할 수 있게 해주는 도구입니다. GPT 나 Claude 보다 성능은 낮지만, 무료로 개념을 학습하기에 적합합니다. `ollama pull` 모델명으로 모델을 받고, LangChain 의 `ChatOllama` 클래스로 호출하면 됩니다.

개념 설명: `deepseek-r1` 같은 모델은 “추론 모델”이므로, 답변 안에 `<think>...</think>` 같은 생각 과정이 포함될 수도 있습니다. `llama3.2` 는 한글을 공식적으로 지원하지 않아 한국어 질문에 대한 답변이 어색하게 나올 수 있으며, 반대로 `exaone` 은 한글을 공식적으로 지원하는 모델입니다. 같은 코드를 사용하더라도 모델에 따라 한국어 품질이 크게 달라질 수 있으니 유의해야 합니다.

```
# ChatOllama 를 이용해 로컬 LLM(deepseek-r1:1.5b) 호출
from langchain_ollama import ChatOllama

llm = ChatOllama(model="deepseek-r1:1.5b")
result = llm.invoke("What is the capital of France?") # 추론 모델이라 생각할
# 만한 질문을 넣어봄
print(result)

# result.content      : AI 의 실제 답변 텍스트
# additional_kwargs   : 추가 정보
# response_metadata   : 모델 실행 상세 정보 (총 소요시간, 모델 로딩 시간, 프롬프
# 트 처리 시간, 답 생성 시간)
# usage_metadata      : 처리한 토큰 수 (input/output/total)

# 한글 미지원 모델(llama3.2:1b)로 한국어 질문 테스트 -> 답이 어색하게 나올 수 있음
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2:1b")
result = llm.invoke("한국 수도는 어디예요?")
print(result.content)

# Llama3.2 는 공식적으로 한글 지원 x (405b, 3.3 70b 는 예외)
# exaone 은 공식적으로 한글 지원 -> 한국어 실습에는 exaone 추천
```

2) OpenAI 활용

요약: langchain-openai 패키지의 ChatOpenAI 로 GPT 모델을 호출할 수 있습니다. .env 파일에 OPENAI_API_KEY 를 넣고 load_dotenv()로 불러오는 방식이 일반적입니다. 코랩에서는 .env 를 사용할 수 없으므로 google.colab.userdata 를 대신 사용합니다.

```
# .env 파일에서 환경변수 불러오기 (로컬 환경 기준)
from dotenv import load_dotenv
load_dotenv() # .env 파일의 OPENAI_API_KEY 등을 환경변수로 등록

from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4.1-nano")
result = llm.invoke("What is the capital of Korea? Answer me in Korean")
print(result.content) # 예: '대한민국의 수도는 서울입니다.'
```

💡 코랩에서는 .env 를 사용할 수 없으므로 아래와 같이 처리합니다:

```
from google.colab import userdata
api_key = userdata.get('OPENAI_API_KEY')
```

3) 다른 벤더(Anthropic 등) 활용

요약: 모든 벤더가 OPENAI_API_KEY 를 사용하는 것은 아닙니다. Claude 는 Anthropic 벤더이므로 ANTHROPIC_API_KEY 가 필요합니다. Azure, Upstage, Bedrock 등도 각자 다른 환경변수 이름을 사용합니다. **에러 메시지를 읽고 어떤 환경변수가 필요한지 확인하는 습관**을 들이는 것이 중요합니다.

```
# Anthropic Claude 모델 호출 (환경변수 ANTHROPIC_API_KEY 필요)
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-3-5-sonnet-20240620")
# llm.invoke("What is the capital of Korea?")
# 환경변수가 없으면 에러 발생 -> langchain 공식문서에서 ANTHROPIC_API_KEY 라는
# 이름의 환경변수를 요구한다는 걸 확인하고 설정해야 함
```

2. 랭체인 스타일로 프롬프트 작성하기

요약: LLM 을 호출할 때 사용하는 질문(프롬프트)에는 3 가지 타입이 있습니다 — (1) 단순 문자열, (2) PromptValue(템플릿에서 생성된 값), (3) BaseMessage 리스트(대화 형태). 이 파트에서는 이 세 가지를 하나씩 다룹니다.

1) 기본 프롬프트 템플릿 사용 (PromptTemplate)

개념 설명: PromptTemplate 은 {country} 같은 placeholder(빈칸)를 사용하여, 나중에 실제 값으로 바꿔 넣을 수 있는 템플릿을 만드는 도구입니다. .invoke({...})를 호출하면 실제 값이 채워진 PromptValue 가 생성됩니다.

```
from langchain_core.prompts import PromptTemplate
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2:1b")

# {country} 자리에 나중에 실제 나라 이름이 들어감 (placeholder)
prompt_template = PromptTemplate(
    template="What is the capital of {country}",
    input_variables=["country"]
)

prompt = prompt_template.invoke({"country": "Korea"}) # 실제 값 주입
print(prompt) # text='What is the capital of Korea'
llm.invoke(prompt) # 완성된 프롬프트로 LLM 호출
```

2) 메시지 기반 프롬프트 작성 (BaseMessage 리스트)

개념 설명: SystemMessage(역할 지정), HumanMessage(사용자 발화), AIMessage(AI 답변)를 리스트로 순서대로 넣으면 대화 히스토리를 만들 수 있습니다. 다만 이 방식은 {country} 같은 placeholder 를 넣어도 실제 값으로 바뀌지 않습니다 — 이 점이 PromptTemplate 과 결정적으로 다른 부분입니다. 이러한 한계를 극복하기 위해 ChatPromptTemplate 을 사용합니다.

```
from langchain_core.messages import AIMessage, HumanMessage, SystemMessage
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2:1b")

# 대화 히스토리를 직접 메시지 리스트로 구성
```

```

message_list = [
    SystemMessage(content="You are a helpful assistant!"),
    HumanMessage(content="What is the capital of Italy?"),
    AIMessage(content="The capital of Italy is Rome."),
    HumanMessage(content="What is the capital of France?"),
    AIMessage(content="The capital of France is Paris."),
    HumanMessage(content="What is the capital of Korea?")
]
llm.invoke(message_list)

# 주의: BaseMessage 리스트에 {country} 같은 placeholder 를 넣어도
# 자동으로 값이 채워지지 않음 (렌제인화 x)

```

3) ChatPromptTemplate 사용

개념 설명: BaseMessage 리스트 대신 튜플 리스트로 ("system", "..."), ("human", "...") 형태로 넣으면, {country} 같은 placeholder 가 실제 값으로 치환됩니다. 이는 실전에서 가장 많이 사용되는 방식입니다.

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2:1b")

# 튜플 리스트로 구성 -> {country} placeholder 가 정상적으로 치환됨
chatPromptTemplate = ChatPromptTemplate.from_messages([
    ("system", "당신은 대한민국 전문 도우미야!"),
    ("human", "{country}의 수도가 어디예요?"),
])

country = input("어느 나라 수도가 궁금하세요?")
prompt = chatPromptTemplate.invoke({"country": country})
print("프롬프트 :", prompt)
result = llm.invoke(prompt)
print(result.content)

```

3. 답변 형식 컨트롤하기 (Output Parser)

요약: `llm.invoke()`의 결과는 기본적으로 `AIMessage` 객체입니다. 하지만 우리가 원하는 것은 보통 순수 문자열, JSON, 혹은 파이썬 객체 하나(예: "Seoul"만)입니다. 이럴 때 **OutputParser** 를 붙여서 원하는 형식으로 변환합니다.

1) 문자열 출력 파서 (StrOutputParser)

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# 명확한 지시사항을 프롬프트에 포함시키는 게 포인트 (도시명만 반환하도록 지시)
prompt_template = PromptTemplate(
    template="What is the capital of {country}. Return the name of the city only",
    input_variables=["country"]
)

prompt = prompt_template.invoke({"country": "Korea"})
ai_message = llm.invoke(prompt) # AIMessage 객체
output_parser = StrOutputParser()
result = output_parser.invoke(ai_message) # 순수 문자열 "Seoul"로 변환
print(result)
```

2) JSON 출력 파서 (JsonOutputParser)

개념 설명: LLM 에 JSON 형식으로 답변을 요청하면 보통 코드블록(```json)까지 함께 출력되는 경우가 많습니다. `JsonOutputParser` 는 이런 응답에서 JSON 만 뽑아서 파이썬 dict 로 변환해 줍니다. 프롬프트에 **예시 형식**을 함께 제시하면 훨씬 안정적인 결과를 얻을 수 있습니다.

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate

country_detail_prompt = PromptTemplate(
    template="""Give following information about {country}
    - Capital
    - Population
    - Language
    - Currency
    Return ONLY a valid JSON object with no additional text
    Example format:
    [{"capital": "Jeju", "population": "2 million", "language": "English", "c
```

```

urrency": "dollar"}}
    """
    input_variables=["country"]
)

output_parser = JsonOutputParser()
prompt = country_detail_prompt.invoke({"country": "Korea"})
ai_message = llm.invoke(prompt)
json_result = output_parser.invoke(ai_message)  # dict 로 변환됨
print(type(json_result), json_result)

```

3) 구조화된 출력 (Pydantic + with_structured_output)

개념 설명: JsonOutputParser 는 LLM 이 JSON 을 다소 어긋난 형태로 출력할 경우 파싱이 실패할 수 있어 불안정합니다. **Pydantic** 의 BaseModel 로 원하는 데이터 구조(필드 이름, 타입, 설명)를 미리 정의해두고, llm.with_structured_output(모델클래스)로 감싸면 훨씬 안정적으로 구조화된 값을 받을 수 있습니다. Field(description=...)에 작성한 설명은 LLM 에게 “이 필드에 무엇을 입력해야 하는지”에 대한 힌트를 제공하여 정확도를 높여줍니다.

```

from pydantic import BaseModel, Field
from langchain_core.prompts import PromptTemplate

# 원하는 응답 구조를 Pydantic 모델로 정의
class CountryDetail(BaseModel):
    capital: str = Field(description="the capital of the country")
    population: int = Field(description="the population of the country")
    language: str = Field(description="the language of the country")
    currency: str = Field(description="the currency of the country")

country_detail_prompt = PromptTemplate(
    template="""Give following information about {country}
    - Capital
    - Population
    - Language
    - Currency
    return in JSON format and return the JSON dictionary only""",
    input_variables=["country"]
)

# LLM 에 구조화된 출력 기능을 장착
structured_llm = llm.with_structured_output(CountryDetail)
info = structured_llm.invoke(country_detail_prompt.invoke({"country": "Korea"}))

```

```

print(info)                # capital='Seoul' population=51 ...
print(info.capital, info.population)  # 필드별로 바로 접근 가능
print(info.model_dump_json())  # JSON 문자열로 변환
print(info.model_dump())      # dict 로 변환

```

4. LCEL(LangChain Expression Language)로 체인 만들기

요약: PromptTemplate, ChatOllama, StrOutputParser 같은 것들은 전부 Runnable 이라는 공통 인터페이스를 상속받고 있어 .invoke()를 사용할 수 있습니다. 이 덕분에 **파이프 연산자(|)** 로 여러 단계를 하나로 연결할 수 있으며, 이것이 LCEL 의 핵심입니다.

1) 단순 체인 구성

```

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2:1b", temperature=0)  # temperature=0 -> 일관된 답변 유도

prompt_template = PromptTemplate(
    template="What is the capital of {country}. Return the name of the city only",
    input_variables=["country"]
)
output_parser = StrOutputParser()

# 프롬프트템플릿 -> llm -> 출력파서를 파이프(|)로 연결 (체인도 Runnable 이라 invoke 가능)
capital_chain = prompt_template | llm | output_parser
result = capital_chain.invoke({"country": "Korea"})
print(result)  # "Seoul"

```

2) 복합 체인 구성 (체인끼리 연결)

개념 설명: 체인끼리도 다시 파이프를 연결할 수 있습니다. “나라 설명 → 나라 이름 → 그 나라 수도”처럼 여러 단계의 추론이 필요할 때는, 체인 하나로 모든 것을 처리하기보다

작은 체인으로 쪼개서 연결하는 습관이 좋습니다. 이렇게 하면 디버깅이 쉬워지고 재사용성도 높아집니다.

```
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough

# 1 단계: 힌트(정보)를 보고 나라 이름을 맞추는 체인
country_prompt = PromptTemplate(
    template="""Guess the name of the country based on the following information:
    {information}
    Return the name of the country only""",
    input_variables=["information"]
)
country_chain = country_prompt | llm | output_parser

# 2 단계 체인(capital_chain)은 위에서 이미 만든 것 재사용
# country_chain의 출력("Italy")이 capital_chain의 입력({"country":...})으로 들어가야 함
# -> dict 형태로 감싸서 키를 맞춰줌
final_chain = {"country": country_chain} | capital_chain

result = final_chain.invoke("This country is very famous for its wine")
print(result) # "Rome"

# RunnablePassthrough : 입력값을 그대로(가공 없이) 다음 단계로 전달해주는 Runnable
# 여러 변수를 동시에 다음 단계로 넘겨야 할 때 사용
final_chain2 = {"information": RunnablePassthrough()} | {"country": country_chain} | capital_chain
result2 = final_chain2.invoke("This country is very famous for its wine")
print(result2) # "Rome"
```

⚠ 주의: 한국어를 공식적으로 지원하지 않는 모델(llama3.2 등)은 체인이 길어질수록 한국어 처리가 더 불안정해집니다. 한국어 실습은 exaone 계열 모델을 사용하는 것을 권장합니다.

5. 생성형 AI 체인 실전 예제: 나라 → 대표 음식 → 레시피

요약: 지금까지 배운 개념(PromptTemplate, temperature, 시스템 메시지, 체인 연결)을 한번에 활용해보는 종합 실습입니다. “나라 이름을 입력하면 → 그 나라의 대표 음식을 찾고 → 그 음식의 레시피까지 자동으로 생성”하는 체인을 만들어봅니다.

1) LLM 설정 — 프롬프트 팁 1: Temperature 조절

```
from langchain_ollama import ChatOllama

# temperature=0 : 매번 같은/일관된 답변을 유도 (창의성보다 정확성이 중요할 때)
llm = ChatOllama(model="llama3.2:1b", temperature=0)
```

2) 음식 추천 체인 — 프롬프트 팁 2: 명확한 출력 형식 지정

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

# "음식 이름만" 반환하라고 명확히 지시 -> 불필요한 설명이 안 붙어서 토큰도 아낌
food_prompt = PromptTemplate(
    template="""what is one of the most popular food in {country}?
    Please return the name of the food only""",
    input_variables=['country']
)

food_chain = food_prompt | llm | StrOutputParser()
result = food_chain.invoke({'country': 'South Korea'})
print(result) # "Bibimbap."
```

3) 레시피 생성 체인 — 프롬프트 팁 3: 시스템 메시지 활용

```
from langchain_core.prompts import ChatPromptTemplate

# 시스템 메시지로 "역할"과 "응답 형식"을 명확히 지정
recipe_prompt = ChatPromptTemplate.from_messages([
    ('system', '''Provide the recipe of the food that the user wants.
    Please return the recipe only as a numbered list without any explanation'''),
    ('human', 'Can you give me the recipe for making {food}?')
])

recipe_chain = recipe_prompt | llm | StrOutputParser()
result = recipe_chain.invoke({'food': 'Bibimbap'})
print(result)
```

4) 체인 연결로 최종 체인 구성

```
# food_chain의 출력(음식 이름)이 recipe_chain의 입력({"food":...})이 되도록 연결
final_chain = food_chain | recipe_chain

# 나라 이름 하나만 넣으면 대표 음식 -> 레시피까지 한 번에 생성됨
recipe = final_chain.invoke("Korea")
print(recipe)
```

5) 한국어 버전 (한글 공식 지원 모델 exaone3.5:2.4b 사용)

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 한글 공식 지원 모델 사용 (deepseek는 추론 과정까지 같이 나눔)
llm = ChatOllama(model="exaone3.5:2.4b", temperature=0)

food_prompt = PromptTemplate(
    template='''{country}에서 가장 인기 있는 음식 중 하나는 무엇인가요?
                음식 이름만 한단어로 반환해 주세요''',
    input_variables=['country']
)
food_chain = food_prompt | llm | StrOutputParser()

recipe_prompt = ChatPromptTemplate.from_messages([
    ('system', '사용자가 원하는 음식의 레시피를 제공합니다. 번호를 달아 설명해 주세요'),
    ('human', '{food}을 만드는 레시피를 알려주세요')
])
recipe_chain = recipe_prompt | llm | StrOutputParser()

final_chain = food_chain | recipe_chain
recipe = final_chain.invoke("한국")
print(recipe)

# 참고: 령체인(체인 연결)을 한국어로 시킬 경우, 여러 단계가 이어지면
# 한글 지원이 약한 모델일수록 오류가 잦음 (단일 호출은 잘 되는 편)
```

📌 핵심 개념 총정리

LLM 호출	: ChatOllama, ChatOpenAI, ChatAnthropic
LLM 호출용 프롬프트	: PromptTemplate, ChatPromptTemplate
결과 형식 변환(Parser)	: StrOutputParser / JsonOutputParser < Pydantic(BaseModel) 기반
	with_structured_output() 이용이 가장 안정적
체인 연결(LCEL)	: 위 요소들이 전부 Runnable 이라 파이프()로 연결 가능

- 다음 시간 예고 키워드: **RAG (Retrieval-Augmented Generation, 검색 증강 생성)**

📝 실습 과제 제안

1. ChatOllama 로 llama3.2:1b 와 exaone3.5:2.4b 에 같은 한국어 질문을 던져서 답변 품질을 비교해보기
2. PromptTemplate + JsonOutputParser 로 “도시 이름을 넣으면 관광지 3 곳을 JSON 으로 반환”하는 체인 만들어보기
3. Pydantic BaseModel 로 나만의 데이터 구조(예: 영화 정보: 제목/감독/연도)를 정의하고 with_structured_output 으로 받아보기
4. food_chain | recipe_chain 처럼, 3 단계 이상 이어지는 나만의 체인을 설계해보기 (예: 나라 → 관광지 → 여행 코스)

⚠️ 학습 중 자주 발생하는 실수 및 오류 포인트

상황	원인
OPENAI_API_KEY 관련 에러	.env 파일 미설정 또는 load_dotenv() 누락
Anthropic 호출 에러	ANTHROPIC_API_KEY 환경변수 이름을 몰라서 발생 → 공식 문서 확인 습관 강조
BaseMessage 리스트에서 {country}가 안 바뀔	ChatPromptTemplate 이 아니라 순수 리스트라서 치환 기능이 없음
한국어 답변이 이상하게 나옴	사용 중인 모델이 한글 공식 미지원 (llama3.2 등) → exaone 계열로 교체